

Simplex

单纯形算法

定义

单纯形法是解决线性规划问题的一个有效的算法。

线性规划就是在一组线性约束条件下，求解目标函数最优解的问题。

线性规划的一般形式

在约束条件下，寻找目标函数 的最大值：

线性规划的可行域

满足线性规划问题约束条件的所有点组成的集合就是线性规划的可行域。若可行域有界（以下主要考虑有界可行域），线性规划问题的目标函数最优解必然在可行域的顶点上达到最优。

单纯形法就是通过设置不同的基向量，经过矩阵的线性变换，求得基可行解（可行域顶点），并判断该解是否最优，否则继续设置另一组基向量，重复执行以上步骤，直到找到最优解。所以，单纯形法的求解过程是一个循环迭代的过程。

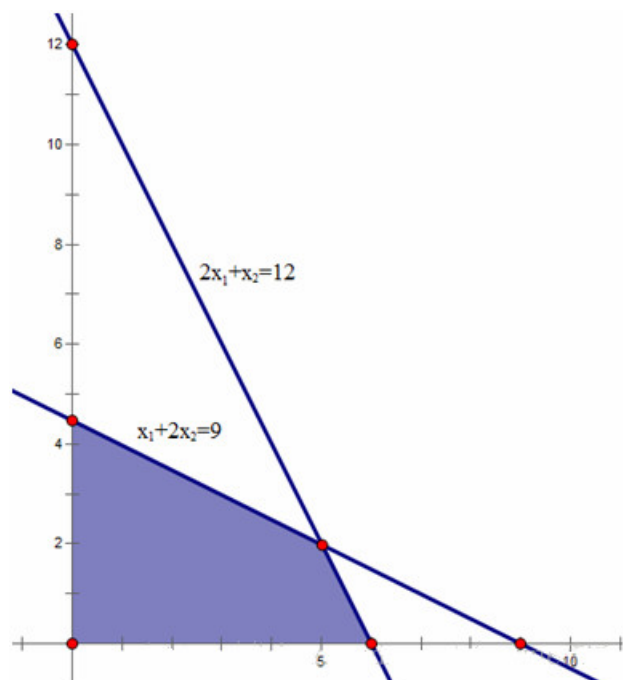


图1 可行域

线性规划的标准形式

在说明单纯形法的原理之前，需要明白线性规划的标准形式。因为单纯形算法是通过线性规划的标准形来求解的。一般，规定线性规划的标准形式为：

写成矩阵形式：

标准形的形式为：

1. 目标函数要求
2. 约束条件均为等式
3. 决策变量为非负约束

普通线性规划化为标准形：

1. 若目标函数为最小化，可以通过取负，求最大化
2. 约束不等式为小于等于不等式，可以在左端加入非负变量，转变为等式，比如：

同理，约束不等式为大于等于不等式时，可以在左端减去一个非负松弛变量，变为等式。

3. 若存在取值无约束的变量，可转变为两个非负变量的差，比如：

本文最开始的线性规划问题转化为标准形为：

单纯形法的思想与例子

几何意义

在标准形中，有 m 个约束条件（不包括非负约束）， n 个决策变量，且 $n > m$ 。首先选取 m 个基变量，基变量对应约束系数矩阵的列向量线性无关。通过矩阵的线性变换，基变量可由非基变量表示：

如果令非基变量等于 0，可求得基变量的值：

如果为可行解的话， $b_i \geq 0$ 。那么它的几何意义是什么呢？

还是通过上述具体的线性规划问题来说明：

如果选择 x_1, x_2 为基变量，那么令 x_3, x_4 等于 0，可以去求解基变量 x_1, x_2 的值。对系数矩阵做行变换，如下所示， x_3, x_4 。

表示可行解在 轴上； 表示可行解在 的直线上。那么，求得的可行解即表示这两条直线的交点，也是可行域的顶点，如图所示：

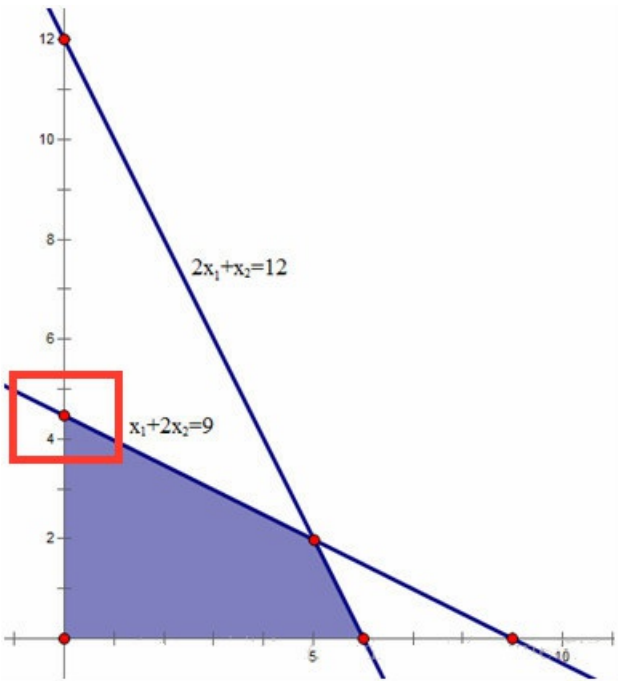


图2

所以，通过选择不同的基变量，可以获得不同的可行域的顶点。

如何判断最优

如前所述，基变量可由非基变量表示：

目标函数 也可以完全由非基变量表示：

当达到最优解时，所有的 应小于等于 ，当存在 ， 时，当前解不是最优解，为什么？

当前的目标函数值为 ，其中所有的非基变量值均取 。由之前分析可知， 代表可行域的某个边界，是 的最小值。如果可行解逐步离开这个边界， 会变大，因为 ，显然目标函数的取值也会变大，所以当前解不是最优解。我们需要寻找新的基变量。

如何选择新的基变量

如果存在多个 ，选择最大的 对应的变量作为基变量，这表示目标函数随着 的增加，增长的最快。

如何选择被替换的基变量

假如我们选择非基变量 作为下一轮的基变量，那么被替换基变量 在下一轮中作为非基变量，等于 。选择 的原则：替换后应该尽量使 值最大（因为上面已分析过，目标函数会随着 的增大而增大），但要保证替换基变量后的解仍是可行解，因此应该选择最紧的限制。

继续通过上面的例子来说明：

从最后一行可以看到， 的系数为 ，所以选 、 为基变量并没有使目标函数达到最优。下一轮选取 作为基变量，替换

、中的某个变量。

第一行是符号

第二行：若 替换 作为基变量， 时，

第三行：若 替换 作为基变量， 时，

尽管替换 后， 的值更大，但将它代入 后会发现 的值为负，不满足约束。从几何的角度来看，选择 和 作为非基变量，得到的解是直线 和 的交点，它在可行域外。因此应该选择 作为非基变量。

终止条件

当目标函数用非基变量的线性组合表示时，所有的系数均不大于 ，则表示目标函数达到最优。

如果，有一个非基变量的系数为 ，其他的均小于 ，表示目标函数的最优解有无穷多个。这是因为目标函数的梯度与某一边界正交，在这个边界上，目标函数的取值均相等，且为最优。

使用单纯形法来求解线性规划，输入单纯形法的松弛形式，是一个大矩阵，第一行为目标函数的系数，且最后一个数字为当前轴值下的 值。下面每一行代表一个约束，数字代表系数每行最后一个数字代表 值。

算法和使用单纯性表求解线性规划相同。

对于线性规划问题：

我们可以得到其松弛形式：

我们可以构造单纯形表，其中最后一行打星的列为轴值。

在单纯形表中，我们发现非轴值的 上的系数大于零，因此可以通过增加这些个 的值，来使目标函数增加。我们可以贪心的选择最大的 ，在上面的例子中我们选择 作为新的轴，加入轴集合中，那么谁该出轴呢？

其实我们由于每个 都大于零，对于 它的增加是有所限制的，如果 过大，由于其他的限制条件，就会使得其他的 小于零，于是我们应该让 一直增大，直到有一个其他的 刚好等于 为止，那么这个 就被换出轴。

我们可以发现，对于约束方程 ，即第一行约束， 最大可以为 ()，对于约束方程 ， 最大可以为 ()，因此 最大只能为他们之间最小的那个，这样才能保证每个 都大于零。因此使用第 行，来对各行进行高斯行变换，使得第二列第四行中的每个 都变成零，也包括 。这样我们就完成了把 入轴， 出轴的过程。变换后的单纯形表为：

继续计算，我们得到：

此时我们发现，所有非轴的 的系数全部小于零，即增大任何非轴的 值并不能使得目标函数最大，从而得到最优解。

单纯形法的具体实现

标准型

个约束 个变量用 向量表示， 是一个 的矩阵， 是一个 的向量， 是一个 的向量，最大化 满足约束。

最大化 满足如下约束条件：

个变量， 个约束，构造 的矩阵， 维向量， 维向量

最大化 满足如下约束条件：

转换为标准型

若目标函数要求取最小值，那么可以对其取相反数变成取最大值。对于限制条件， 可以用两个不等式 描述，对于限制条件， 可以用不等式 描述。对于无限制的变量， 可以将其拆为两个非负变量， 使得。

松弛型

基本变量， ， 一个约束对应一个， 表示松弛量，叫做松弛变量（基本变量）

非基变量， ，

松弛变量

等式左侧为基本变量，右侧为非基本变量。

变量

- 替入变量（非基变量）
- 替出变量（基本变量）

可行解

- 基本解：所有非基变量设为 0，基本变量为右侧的常数
- 基本可行解：所有

注：单纯形法的过程中 θ 和 ρ 不断交换，在 n 维空间中不断走，「相当于不等式上的高斯消元」。

转轴

选取一个非基本变量 x_{enter} 为替入变量，基本变量 x_{leave} 为替出变量，将其互换，为了防止循环，根据 **Bland 规则**，选择下标最小的变量。

Bland 规则 可以参看：[最优化方法](#)

初始化

在所有 b_i 的约束中随机选一个作为 x_{enter} ，再随机选一个 x_{leave} 作为 x_{leave} ，然后 θ 就变正了。

如果不存在这样的 θ ，那么说明原问题无解。

算法实现

每个约束定义了 n 维空间中的一个半空间（超平面），交集形成的可行域是一个凸区域称为单纯形。目标函数是一个超平面，最优解在凸区域定点处取得。通过不断的转轴操作，在 n 维凸区域的顶点上不断移动（转轴），使得基本解的目标值不断变大，最终达到最优解。如果基本解的目标值没有上限，那么说明原问题无界。

以下问题可以转换为单纯形：

- 最短路
- 最大流
- 最小费用最大流
- 多商品流

基本思想就是改写 a_{ij} 这个约束为 x_{enter} 作为基本变量，然后把这个新 θ 的值带到其他约束和目标函数中，就消去 x_{enter} 了。改写和带入时要修改 b_i 和 c_j ，目标函数则是 c_j 和 b_i 。

转动时， θ 和 ρ 并没有像算法导论上一样，矩阵用了两行分别是 θ 和 ρ （这样占用内存大），而是用了同一行，这样矩阵的行数，列数。

也就是说，约束条件只用 n 个，尽管 θ 和 ρ 不断交换，但同一时间还是只有 n 个约束（基本变量）， n 个非基变量，注意改写成松弛型后矩阵实际系数为负。（一个优化为 $\leq$ 的约束没必要带入了）。

simplex 是主过程，基本思想是找到一个 θ 的，然后找对这个 θ 限制最紧的，转动这组 θ ，注意精度控制， ϵ ，还有找 ρ 的时候才行。

► 例题 [「NOI2008」志愿者招募](#)

原始问题 n 个变量， m 个约束，当 $b_i \geq 0$ ，。

对偶问题 m 个变量， n 个约束

把对应出的系数矩阵代入到单纯形算法就可以求出最优解了。

```
1 #include <LTcmath>
2 #include <LTcstring>
3 #include <LTiostream>
4 using namespace std;
5 constexpr int M = 10005, N = 1005, INF = 1e9;
6
7 int n, m;
8 double a[M][N], b[M], c[N], v;
9
10 void pivot(int l, int e) { // 转轴操作函数
11     b[l] /= a[l][e];
12     for (int j = 1; j <= n; j++)
13         if (j != e) a[l][j] /= a[l][e];
14     a[l][e] = 1 / a[l][e];
15
16     for (int i = 1; i <= m; i++)
17         if (i != l && fabs(a[i][e]) > 0) {
18             b[i] -= a[i][e] * b[l];
19             for (int j = 1; j <= n; j++)
20                 if (j != e) a[i][j] -= a[i][e] * a[l][j];
21             a[i][e] = -a[i][e] * a[l][e];
22         }
23
24     v += c[e] * b[l];
25     for (int j = 1; j <= n; j++)
26         if (j != e) c[j] -= c[e] * a[l][j];
27     c[e] = -c[e] * a[l][e];
28
29     // swap(B[l],N[e])
30 }
31
32 double simplex() {
33     while (true) {
34         int e = 0, l = 0;
35         for (e = 1; e <= n; e++)
36             if (c[e] > (double)0) break;
37         if (e == n + 1) return v; // 此时v即为最优解
38         double mn = INF;
39         for (int i = 1; i <= m; i++) {
40             if (a[i][e] > (double)0 && mn > b[i] / a[i][e]) {
41                 mn = b[i] / a[i][e]; // 找对这个e限制最紧的l
42                 l = i;
43             }
44         }
45         if (mn == INF) return INF; // unbounded
46         pivot(l, e); // 转动l,e
47     }
48 }
49
50 int main() {
51     cin.tie(nullptr)->sync_with_stdio(false);
52     cin >> n >> m;
53     for (int i = 1; i <= n; i++) cin >> c[i];
54     for (int i = 1; i <= m; i++) {
55         int s, t;
56         cin >> s >> t;
57         for (int j = s; j <= t; j++) a[i][j] = 1; // 表示第i种志愿者在j时间可以服务
58         cin >> b[i];
59     }
60     cout << (int)(simplex() + 0.5);
61 }
```

对偶原理

最大化与最小化互换，常数与目标函数互换，改变不等号，变量与约束对应。

表示 是否匹配

令 为两类约束对偶之后的变量

全幺模矩阵 (Totally Unimodular Matrix)

当一个矩阵的任意一个子方阵的行列式都为 1 或 -1 时，我们称这个矩阵是全幺模的。

如果单纯形矩阵是全幺模的，那么单纯形就具有整数解。

▼ 证明

在线性规划中，最优解通常位于由约束条件形成的可行域的边界上。具体来说，它位于这个高维多面体的顶点处。这些顶点可以通过将部分线性独立的不等式约束转化为等式，然后求解得到的线性方程组来确定。如果约束矩阵是全幺模的（即每个方阵子矩阵的行列式为 1 或 -1），那么根据克莱姆法则（Cramer's rule），这些线性方程组的解将是整数。

线性规划中 为全幺模矩阵，则单纯形法过程中所有系数，可以去除系数为 1 的项进行优化！

注：任何最大流、最小费用最大流的线性规划都是全幺模矩阵

习题练习

- [UOJ#179. 线性规划](#)

参考资料

- [线性规划之单纯形法【超详解 + 图解】](#)
- [2016 国家集训队论文](#)
- [算法导论](#)

本页面最近更新：2024/6/25 19:14:29, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [AngelKitty](#), [Back1ght](#), [baker221](#), [CCXXI](#), [Chrogeek](#), [countercurrent-time](#), [Early0v0](#), [Enter-tainer](#), [FFjet](#), [GaisaiYuno](#), [Gesruea](#), [H-J-Granger](#), [Henry-ZHR](#), [HeRaNO](#), [HomuraCat](#), [iamtwz](#), [ImpleLee](#), [Ir1d](#), [kenlig](#), [ksyx](#), [lychees](#), [Marcythm](#), [MegaOwler](#), [Molmin](#), [NachtgeistW](#), [opsiff](#), [orzzzjq](#), [ouuan](#), [Raisetsu41](#), [sshwy](#), [StudyingFather](#), [SukkaW](#), [Tiphereth-A](#), [Xeonacid](#), [ZnPdCo](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用